GILBERT HARA

BScICT/24/200/015

1.Pointer Arithmetic Demo in C++

```
#include <stdio.h>

int main(void) {

   int num = 7;

   int *ptr = NULL;

   ptr = &num;

   printf("ptr-1 : %p \n", (void *)(ptr - 1));

   printf("ptr   : %p \n", (void *)ptr);

   printf("ptr+1 : %p \n\n", (void *)(ptr + 1));

   printf("size of data type : %lu \n\n", sizeof(num));

   return 0;

}
```

Overview

This program demonstrates how pointer arithmetic works in C. It shows how incrementing or decrementing a pointer moves it by the size of the data type it points to.

How It Works

1. An integer `num` is declared and initialized to `7`.
2. A pointer `ptr` is assigned the address of `num`.
3. The program prints:
   - `ptr - 1`: the address one integer before `num`
   - `ptr`: the actual address of `num`
   - `ptr + 1`: the address one integer after `num`
4. Finally, it prints the size of the `int` data type on the system.

```
ptr-1 : 0x7ffee4b8c8a8
ptr   : 0x7ffee4b8c8ac
ptr+1 : 0x7ffee4b8c8b0

size of data type : 4
```

2.Pointer Arithmetic with `char` in C++

```
#include <stdio.h>

int main(void) {

    char c = 'a';

    char *ptr = NULL;

    ptr = &c;

    printf("ptr-1 : %p \n", (void *)(ptr - 1));

    printf("ptr   : %p \n", (void *)ptr);

    printf("ptr+1 : %p \n\n", (void *)(ptr + 1));

    printf("size of data type : %lu \n\n", sizeof(c));

    return 0;

}
```

Overview

This program demonstrates pointer arithmetic using the **`char`** data type in C. It shows how incrementing or decrementing a pointer moves it by the size of the type it points to — in this case, just **1 byte**.

How It Works

1. A character **`c`** is declared and initialized to **`'a'`.**
2. A pointer **`ptr`** is assigned the address of **`c`**.
3. The program prints:
    - **`ptr - 1`:** the address one character before **`c`**
    - **`ptr`:** the actual address of **`c`**
    - **`ptr + 1`:** the address one character after **`c`**
4. Finally, it prints the size of the **`char`** data type on the system.

```
ptr-1 : 0x7ffee4b8c8ab
ptr   : 0x7ffee4b8c8ac
ptr+1 : 0x7ffee4b8c8ad

size of data type : 1
```

Notice how:

- `ptr-1` is **1 byte less** than **`ptr`**
- `ptr+1` is **1 byte more** than **`ptr`**
- The difference matches **`sizeof`**`(`**`char`**`)`

3.Printing Variable and Address in C++

```
#include <stdio.h>

int main(void){

    char c = 'a';

    printf("c : %c\n", c);

    printf("&c : %p\n\n", (void *)&c);

    return 0;

}
```

Overview

This program demonstrates how to print both the value of a variable and its memory address in C. It uses the **`%c`** format specifier to display a character and **`%p`** to correctly show its address.

How It Works

1. A character **`c`** is declared and initialized to **`'a'`.**
2. The program prints:
    - The value of `c` using **`%c`**
    - The memory address of `c` using **`%p`**
3. This highlights the difference between the **data stored** and the **location in memory** where it resides.

```
c : a
&c : 0x7ffee4b8c8ac
```

- The first line shows the character stored in `c`.
- The second line shows the memory address where `c` is stored.

4.Array and Pointer Arithmetic in C++

```
#include <stdio.h>

int main(void) {

   int arr[3] = {7, 14, 21};

   int *ptr = arr;        // ptr points to arr[0]

   printf("%d \n", ptr[1]); // prints 14

   printf("%d \n", arr[1]); // prints 14

   // end of main

}
```

Overview

This program demonstrates how arrays and pointers are closely related in **C**. It shows that accessing array elements using pointer arithmetic (**`ptr`**`[index]`) is equivalent to using array indexing (**`arr`**`[index]`).

How It Works

1. An integer array **`arr`** of size **3** is declared and initialized with values **`{7, 14, 21}`.**
2. A pointer **`ptr`** is assigned to point to the first element of the array **(`arr[0]`).**
3. The program prints:
   - **`ptr[1]`:** the second element of the array **(value `14`)**
   - **`arr[1]`:** the same second element of the array **(value `14`)**

This demonstrates that **`ptr[index]`** and **`arr[index]`** are interchangeable when **`ptr`** points to the start of the array.

```
14
14
```

Both lines print the same value because:

- `ptr[1]` means ***(ptr + 1),** which is the second element of the array.
- `arr[1]` directly accesses the second element of the array.

5.Pointer Arithmetic with Negative Indexing in C++

```
#include <stdio.h>

int main(void) {

   char tmp[5] = {1, 2, 3, 4, 5};

   char *arr = tmp + 4;     // arr now points to tmp[4] (value 5)

   printf("%d \n", arr[-2]); // This is allowed! Prints 3

   // end of main

}
```

Overview

This program demonstrates how pointer arithmetic allows both positive and negative indexing in C. By moving a pointer to a specific element of an array, you can access earlier or later elements using offsets.

How It Works

1. A character array `tmp[5]` is initialized with values `{1, 2, 3, 4, 5}.`
2. A pointer `arr` is set to point to `tmp[4]` (the last element, value 5).
3. The program prints `arr[-2],` which moves the pointer two positions back and accesses `tmp[2]` (value 3).

This shows that pointer indexing works like `*(arr + offset),` and offsets can be negative.

```
3
```

- `arr[-2]` → `*(arr - 2)` → `tmp[2]` → value **3.**

Modifying Arrays with Functions in C++

```
#include <stdio.h>

void addNumber(int *ptr) {     // receives address of array

   int i = 0;

   for(i = 0; i < 10; i++) {

      ptr[i] += 100;         // add 100 to each element

   }

}

int main(void) {

   int i = 0;

   int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

   printf(">> Before function call <<\n");

   for(i = 0; i < 10; i++) {

      printf("%d ", arr[i]);

   }

   printf("\n\n")

   addNumber(arr);            // array name is address ? call by reference

   printf(">> After function call <<\n");

   for(i = 0; i < 10; i++) {

      printf("%d ", arr[i]);

   }

   // end of main

}
```

Overview

This program demonstrates how arrays can be modified inside a function using pointers. Since arrays in C are passed by reference (their name represents the address of the first element), changes made inside the function affect the original array.

How It Works

1. An integer array `arr[10]` is initialized with values `{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}`.
2. The program prints the array **before** the function call.
3. The function `addNumber(int *ptr)` receives the address of the array and adds `100` to each element.
4. The program prints the array **after** the function call, showing that the elements have been modified.

```
>> Before function call <<
1 2 3 4 5 6 7 8 9 10

>> After function call <<
101 102 103 104 105 106 107 108 109 110
```

• Before the function call, the array contains its original values.

• After the function call, each element has been increased by 100.

6.Array Indexing and Pointer Arithmetic in C++

```
#include <stdio.h>

int main(void) {

   int arr[3] = {7, 14, 21};

   printf("%d \n", arr[2]);     // prints 21

   printf("%d \n", *(arr + 2)); // prints 21

   // end of main

}
```

Overview

This program demonstrates that accessing array elements using the **index operator (`arr[index]`)** is equivalent to using **pointer arithmetic (`*(arr + index)`).**

How It Works

1. An integer array `arr[3]` is declared and initialized with values `{7, 14, 21}`.
2. The program prints:
   - `arr[2]`: the third element of the array (**value** `21`).
   - `*(arr + 2)`: pointer arithmetic that also accesses the third element (**value** `21`).

Both approaches yield the same result because:

- `arr[index]` is defined as `*(arr + index)` in C.

```
21
21
```

- The first line comes from direct array indexing.

- The second line comes from pointer arithmetic, but both point to the same element.

7.Function Calls and Addresses in C++

```
#include <stdio.h>

void funcOne(void) {

   printf("funcOne was called!\n");

} // end of funcOne

void funcTwo(void) {

   printf("funcTwo was called!\n");

} // end of funcTwo

int main(void) {

   funcOne();

   funcTwo();

   printf("Address of funcOne : %p \n", (void *)funcOne);

   printf("Address of funcTwo : %p \n", (void *)funcTwo);
```

```
} // end of main
```

Overview

This program demonstrates two important concepts in C:

1. How to define and call functions (`funcOne` and `funcTwo`).
2. How functions themselves have memory addresses, which can be printed and even manipulated like pointers.

How It Works

1. Two functions are defined:
   - `funcOne()` prints `"funcOne was called!".`
   - `funcTwo()` prints `"funcTwo was called!".`
2. In `main(),` both functions are called.
3. The program then prints the **addresses of the functions**. In C, the function name itself represents the address of the function's entry point in memory.

In your original code, you used `%0x` to print the function addresses. The correct format specifier for pointers is `%p.` Using `%0x` may compile but is not portable or safe. Always cast to `(void *)` and **use** `%p.`

```
funcOne was called!
funcTwo was called!
Address of funcOne : 0x1043f0f10
Address of funcTwo : 0x1043f0f30
```

• The first two lines confirm the functions were executed.

• The last two lines show the memory addresses where the functions are stored.

8.Function Pointers in C++

```
#include <stdio.h>

int adder(int n1, int n2) {

   return n1 + n2;

} // end of adder

int main(void) {

   int (*fPtr)(int, int);   // Function pointer declaration
```

```
    fPtr = adder;           // Assign function address to pointer

    printf("Using a name of function : %d \n", adder(10, 20));

    printf("Using a function pointer : %d \n", fPtr(10, 20));

    printf("\n");

} // end of main
```

Overview

This program demonstrates how to declare and use **function pointers** in C. Function pointers allow you to store the address of a function and call it indirectly, which is useful for callbacks, dynamic function selection, and implementing tables of functions.

How It Works

1. A function **`adder(int n1, int n2)`** is defined to return the sum of two integers.
2. In **`main()`**, a function pointer **`fPtr`** is declared with the same signature as `adder`.
3. The pointer **`fPtr`** is assigned the address of `adder`.
4. The program calls the function in two ways:
    - Directly using the function name (**`adder(10, 20)`**).
    - Indirectly using the function pointer (**`fPtr(10, 20)`**).

Both calls produce the same result.

```
Using a name of function : 30
Using a function pointer : 30
```

- The first line shows the result of calling **`adder`** directly.

- The second line shows the result of calling **`adder`** through the function pointer.

Function Pointers with Multiple Functions in C++

```
#include <stdio.h>

int adder(int n1, int n2) {

    printf("Adder was called : ");

    return n1 + n2;
```

```
} // end of adder

int divider(int n1, int n2) {

    printf("Divider was called : ");

    return n1 / n2;

} // end of divider



void printResult(int n1, int n2, int (*fPtr)(int, int)) {

    printf("%d \n", fPtr(n1, n2));

} // end of printResult

int main(void) {

    int num1 = 10;

    int num2 = 2;

    printResult(num1, num2, adder);

    printResult(num1, num2, divider);

    printf("\n");

} // end of main
```

Overview

This program demonstrates how to use **function pointers** to call different functions dynamically. By passing a function pointer as an argument to another function, you can decide at runtime which function should be executed. This is a common technique for implementing callbacks and flexible function dispatch.

How It Works

1. Two functions are defined:
    - **`adder(int n1, int n2)`** → prints `"Adder was called : "` and returns the sum.
    - **`divider(int n1, int n2)`** → prints `"Divider was called : "` and returns the quotient.
2. A helper function **`printResult(int n1, int n2, int (*fPtr)(int, int)`**) is defined:
    - It accepts two integers and a function pointer.

    - It calls the function pointer with the integers and prints the result.
3. In `main()`:
    - **`printResult(num1, num2, adder)`** calls the `adder` function via pointer.
    - **`printResult(num1, num2, divider)`** calls the `divider` function via pointer.

This shows how the same function (**`printResult`**) can execute different logic depending on the function pointer passed in.

```
Adder was called : 12
Divider was called : 5
```

- The first line shows the sum of `10 + 2`.

- The second line shows the quotient of `10 / 2`.

9.Returning Function Pointers in C++

```
#include <stdio.h>

void func1(void) {

   printf("func1 was called!\n");

} // end of func1

void func2(void) {

   printf("func2 was called!\n");

} // end of func2

// Function that returns a function pointer

void (*whatFunction(int sel))(void) {

   if (sel == 1)

      return func1;

   else if (sel == 2)

      return func2;

} // end of whatFunction

int main(void) {
```

```
    void (*fPtr)(void);

    fPtr = whatFunction(1);

    fPtr();                 // Calls func1

    fPtr = whatFunction(2);

    fPtr();                 // Calls func2



    printf("\n");

} // end of main
```

Overview

This program shows how a function can **return a function pointer** in C. This allows you to dynamically select which function to call at runtime, based on conditions or input values. It's a powerful technique for building flexible and modular code.

How It Works

1. Two functions are defined:
   - **`func1()`** → prints `"func1 was called!"`.
   - **`func2()`** → prints `"func2 was called!"`.
2. A function **`whatFunction(int sel)`** is defined:
   - It returns a pointer to either `func1` or `func2` depending on the value of `sel`.
   - If **`sel == 1`**, it returns `func1`.
   - If **`sel == 2`**, it returns `func2`.
3. **In `main()`:**
   - A function pointer `fPtr` is declared.
   - **`fPtr = whatFunction(1);`** assigns the address of `func1` **to `fPtr`,** then **`fPtr`**`();` calls `func1`.
   - **`fPtr = whatFunction(2);`** assigns the address of `func2` to `f`**`Ptr`,** then **`fPtr`**`();` calls `func2`.

This demonstrates how functions can be chosen dynamically at runtime.

```
func1 was called!
func2 was called!
```

• The first line comes from calling `func1` via the returned pointer.

• The second line comes from calling `func2` via the returned pointer.

10. Pointer to Constant in C++

```
#include <stdio.h>

int main() {

    int num = 20;

    const int *ptr = &num;     // Pointer to constant integer

    // *ptr = 30;     // Error (X) - Cannot change value through pointer

    num = 40;          // Allowed (O) - Can change using variable name

    printf("num = %d\n", num);

    return 0;

}
```

Overview

This program illustrates how a **pointer to a constant** works in C. When you declare a pointer as `const int *ptr`, you cannot modify the value being pointed to through the pointer. However, you can still change the value directly using the variable name.

How It Works

1. An integer **`num`** is declared and initialized to **`20`**.
2. A pointer **`ptr`** is declared as **`const int *ptr`** and assigned the address of `num`.
   - This means `ptr` points to a constant integer — you cannot change the value of `num` through `ptr`.
3. Attempting `*ptr = 30;` would cause a **compile-time error**, because the pointer is to a constant.
4. Modifying `num` directly (`num = 40;`) is allowed.
5. The program prints the updated value of `num`.

```
num = 40
```

• The pointer **`ptr`** prevents modification of `num` through dereferencing.

• The variable **`num`** itself can still be modified directly.

11. Constant Pointer in C++

```
#include <stdio.h>

int main() {

    int num1 = 20;

    int num2 = 30;

    int *const ptr = &num1;    // Constant pointer



    *ptr = 40;        // Allowed (O) - Can change the value it points to

    // ptr = &num2;   // Error (X) - Cannot change the pointer itself

    printf("Value pointed by ptr = %d\n", *ptr);

    return 0;

}
```

Overview

This program illustrates how a **constant pointer** works in C. When you declare a pointer as **`int *const ptr`**, the pointer itself cannot be changed to point to another variable, but the value stored at the memory location it points to **can** be modified.

How It Works

1. Two integers are declared: `num1 = 20` and `num2 = 30`.
2. A constant pointer **`ptr`** is initialized to point to `num1`.
   - Because **`ptr`** is constant, it must always point to `num1`.
3. The program modifies the value of `num1` through the pointer **(`*ptr = 40;`),** which is allowed.
4. Attempting to reassign the pointer (**`ptr = &num2;`**) would cause a **compile-time error**, since the pointer itself is constant.
5. The program prints the value pointed to by **`ptr`**.

```
Value pointed by ptr = 40
```

• The pointer **`ptr`** always points to `num1`.

- The value of `num1` can be changed through **`ptr`.**
- The pointer itself cannot be redirected to another variable.

12. Constant Pointer to Constant Data in C++

```
#include <stdio.h>

int main() {

   int num1 = 10;

   int num2 = 50;

   const int *const ptr = &num1;   // Constant pointer to constant data

   // *ptr = 40;     // Error (X)

   // ptr = &num2;   // Error (X)

   printf("Value = %d\n", *ptr);

   return 0;

}
```

Overview

This program illustrates how a **constant pointer to constant data** works in C. When you declare a pointer as `const int *const ptr`, both the pointer itself and the data it points to are immutable through that pointer. This enforces strict immutability and prevents accidental modification.

How It Works

1. Two integers are declared: `num1 = 10` and `num2 = 50`.
2. A constant pointer to constant data `const int *const ptr` is initialized to point to `num1`.
   - Because the pointer is constant, it must always point to `num1`.
   - Because the data is constant, you cannot modify `num1` through the pointer.
3. Attempting either:
   - `*ptr = 40;` → **compile-time error** (cannot change the value through the pointer).
   - `ptr = &num2;` → **compile-time error** (cannot change the pointer itself).

4. The program prints the value pointed to by `ptr`.

```
Value = 10
```

- The pointer **`ptr`** always points to `num1`.
- Neither the pointer nor the value can be changed through **`ptr`**.

13. Pointer Dereference in C++

```
#include <stdio.h>



int main(void) {

   char c = 'a';

   char *ptr = NULL;

   ptr = &c;

   printf("ptr : %p \n", (void *)ptr);

   printf("*ptr : %c \n\n", *ptr);

   return 0;

}
```

Overview

This program illustrates how pointers can be used to access both the **address** and the **value** of a variable in C. It shows the difference between printing the pointer itself and dereferencing it to get the stored value.

How It Works

1. A character variable `c` is declared and initialized to `'a'`.
2. A pointer `ptr` is declared and assigned the address of `c`.
3. The program prints:
   - `ptr` → the memory address of `c`.
   - `*ptr` → the value stored at that memory address (`'a'`).

This demonstrates the fundamental concept of **dereferencing** a pointer.

```
ptr : 0x7ffee4b8c8ac
*ptr : a
```

- The first line shows the memory address of `c`.
- The second line shows the value stored at that address (`'a'`).

14. Call by Value in C++

```
#include <stdio.h>

// Function that attempts to swap two numbers (Call by Value)

void swap(int n1, int n2) {

    int temp = n1;

    n1 = n2;

    n2 = temp;

    // Changes are local only - original variables in main() are unaffected

}

int main() {

    int num1 = 10;

    int num2 = 50;

    printf(">> Before function call <<\n");

    printf("num1 : %d\n", num1);

    printf("num2 : %d\n\n", num2);

    swap(num1, num2);        // Call by value - copies are passed

    printf(">> After function call <<\n");

    printf("num1 : %d\n", num1);

    printf("num2 : %d\n", num2);

    return 0;

}
```

Overview

This program illustrates the concept of **call by value** in C. When arguments are passed to a function by value, copies of the variables are made. Any changes inside the function affect only the local copies, not the original variables in the calling function.

How It Works

1. Two integers are declared: `num1 = 10` and `num2 = 50`.
2. A function `swap(int n1, int n2)` is defined to swap two numbers:
   - It uses a temporary variable to exchange values.
   - However, since the function receives **copies** of `num1` and `num2`, the original variables remain unchanged.
3. In `main()`:
   - The program prints the values of `num1` and `num2` before the function call.
   - Calls `swap(num1, num2)`.
   - Prints the values again after the function call, showing they are unchanged.

```
>> Before function call <<
num1 : 10
num2 : 50

>> After function call <<
num1 : 10
num2 : 50
```

- The values remain unchanged because the swap function only modified **local copies** of the variables.

15. Void Pointers in C++

```
#include <stdio.h>

int main() {

    int num = 100;

    // Void pointer can point to any data type

    void *ptr = &num;

    // Correct way - must cast before dereferencing

    printf("Value using void pointer = %d\n", *(int *)ptr);

    // Wrong ways (these will cause compile errors):
```

```
    // printf("%d\n", *ptr);          // Error - cannot dereference void*

    // double *dptr = &num;           // Error - type mismatch

    return 0;

}
```

Overview

This program illustrates how **void pointers** work in C. A void pointer (`void *`) is a special type of pointer that can hold the address of any data type. However, since it has no associated type, it cannot be directly dereferenced without casting.

How It Works

1. An integer `num` is declared and initialized to `100`.
2. A void pointer `ptr` is assigned the address of `num`.
   - Void pointers are type-agnostic, so they can point to any variable.
3. To access the value stored at the address, the pointer must be **cast** to the correct type (`(int *)` in this case).
4. The program prints the value using the dereferenced casted pointer.
5. Examples of incorrect usage are shown as comments:
   - Dereferencing without casting (`*ptr`) → compile-time error.
   - Assigning an integer address to a `double *` → type mismatch error.

```
Value using void pointer = 100
```

- The void pointer **`ptr`** successfully points to **`num`**.

- Casting to **`(int *)`** allows proper dereferencing.